

LOCAL LLMS · YOUR HARDWARE · YOUR DATA

4 000 000 Files

40 Knobs

1 Right Answer.

LLAMATOASTER — THE APPLIANCE THAT FINDS IT.

4 000 000 files. 40 knobs. One right answer.

Local inference shouldn't require becoming a llama.cpp expert.

The Problem

01 One bad knob — silent quality loss, or a hard crash

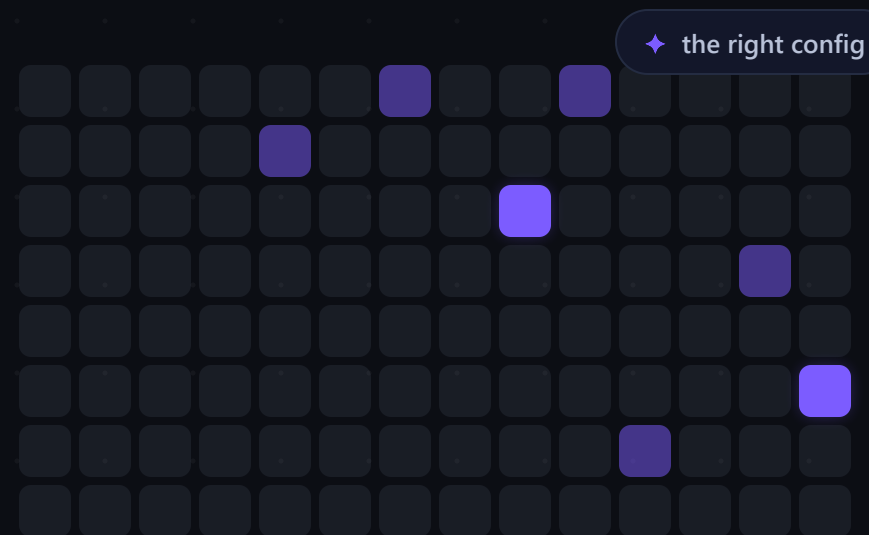
Wrong KV cache → degraded tokens. Context too large → OOM.

02 One good config on GPU X — broken on GPU Y

Reddit recipes fail across architectures, drivers, OS versions.

03 The stale oracle

Ask ChatGPT “best local LLM 2026” → confident, outdated folklore.



~200 000 GGUF ready

≈4 000 000 files

10+ architectures

The Missing Half

- LMArena · Open LLM Leaderboard · MMLU · GPQA · HumanEval · **MT-Bench** — every release is scored nightly.
- But almost nothing answers the other half: **tok/s · latency · genuinely usable context — on YOUR hardware.**

Millions of files and mountains of quality scores — while performance is still folklore and trial-and-error.

QUALITY — SCORED NIGHTLY

every release



answers for
your hardware

tok/s

latency

usable context

OOM-free

Why Now

Qwen3.8 27B $\approx$ previous-generation flagship quality — on an ordinary PC.

- ✗ **Blindly chasing the biggest model**
→ swap, crash, 0.3 tok/s
- ✗ **Chasing hype**
→ Llama 4, when Qwen3.8 is better for YOUR task
- ✗ **Following last month's advice**
→ already stale

Proprietary APIs: outages, refusals, price changes — nothing you control.
Your data stays on your machine.



Gemma 3

Kimi K2

GLM-4

every month a new “impossible” small model

The Fix

One command connects any GPU or CPU box — **pull-only**, **zero open ports**, no firewall gymnastics.

- You state intent: **goal × workload × target context**.
- The grid builds itself. **The sweep runs itself.**

5 min set up tonight

Intent in · **Decision out**

INTENT

goal × workload × target context



GRID + SWEEP

the grid builds itself
the sweep runs itself

Max Speed

lowest latency
for your GPU

Balanced

speed × quality
sweet spot

Max Context

longest verified
context ceiling

Low Memory

runs on modest
hardware

verified context ceilings

concurrency knees

fair model-vs-model

Trust the Card

- ✓ **Eligibility gates.** stability, suspect samples, stddev floors — timer-bug results like **1e6 tok/s** go to the trash, not the average.
- ✓ **Memory from real placement.** per-tensor GGUF placement catches “31/31 offloaded” while secretly running in system RAM.
- ✓ **Fair by construction.** a different machine, build, or GPU rejects the member, not the verdict.
- ✓ **Tamper-evident.** a hash per row, methodology versions never mixed.

Profile · Balanced

RTX 4090 · ggml-v6

tok/s **42.1**

context ceiling **32 768**

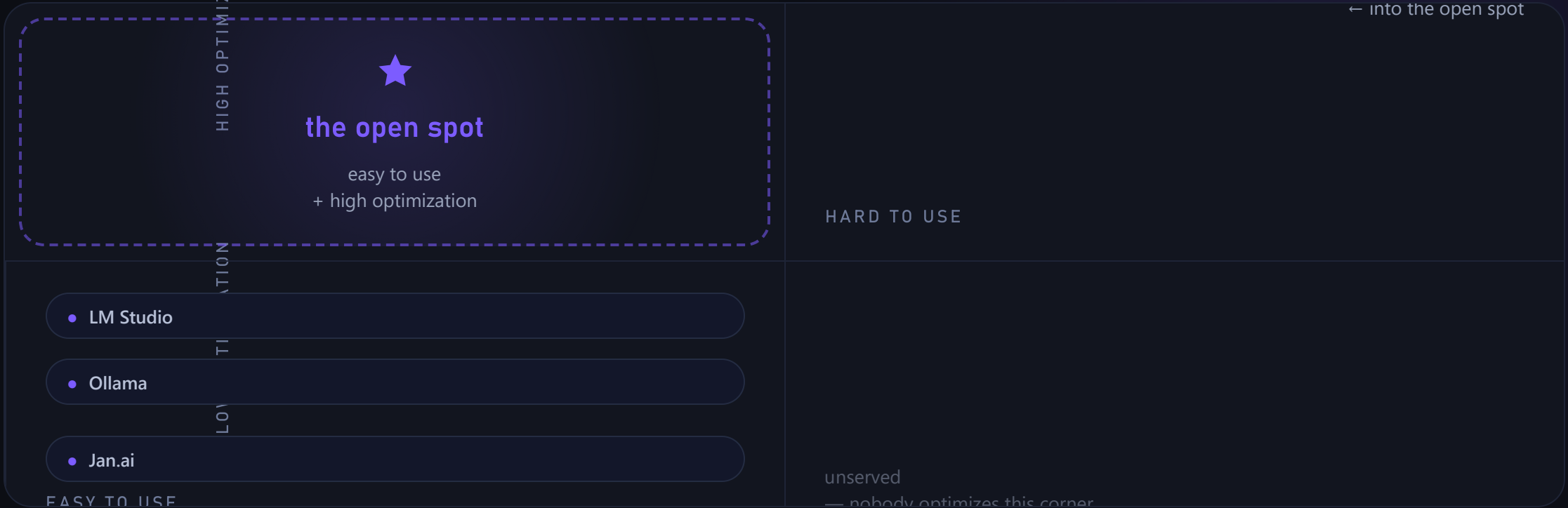
peak memory **11.2 GB**

burst “tok/s” 1e6 **REJECTED**

method v2 · row-hash **d4f9·3c81**



The Competitors



WHAT THEY DO

- ✓ pick a model that “probably works”
- ✓ abstract complexity · one-click install

WHAT THEY DON'T DO

- ✗ find **optimal config** for YOUR hardware
- ✗ **sweep** parameter combinations
- ✗ detect **silent quality degradation**
- ✗ community grid · **tamper-evident** benchmark cards

1 The Ask

command
connects your machine

[llamatoaster.com/ benchmark](https://llamatoaster.com/benchmark)

01 / 03

Run your rig tonight

no port forwarding, no manual build — llama.cpp
installs from the web UI

02 / 03

Join the community grid

opt into the k-anonymized set → the whole catalog
becomes searchable for everyone

03 / 03

Star it while it's early

llamatoaster.com/benchmark — one command
connects your machine